Rel2SQL — Accepted Rel Language Specification

A reverse-engineered grammar and semantics reference, derived from the rel2sql source tree

September 2026

Contents

1 Purpose and Scope **3**

2 Program Structure **4**

3 Formal Grammar **5**
- 3.1 Top level . . . 5
- 3.2 Expressions . . . 5
- 3.3 Formulas . . . 5
- 3.4 Terms (arithmetic — used only inside comparisons and as application arguments' numeric core) . . . 5
- 3.5 Applications, bindings, literals . . . 6

4 Construct-by-Construct Semantics **7**
- 4.1 Full application — `R(x, y, ...)` . . . 7
- 4.2 Partial application — `R[x]`, `R[1, y]` . . . 7
- 4.3 Conjunction / disjunction / negation . . . 7
- 4.4 Quantification — `exists`, `forall` . . . 7
- 4.5 Comparisons . . . 7
- 4.6 Conditional expression — `where` . . . 8
- 4.7 Tuples and products — `(e1, e2, ...)` . . . 8
- 4.8 Relational abstraction (union) — `{e1; e2; ...}` . . . 8
- 4.9 Bindings — `(x): formula` and `[x in D]: expr` . . . 8
- 4.10 Aggregation — `sum[R]`, `average[R]`, `min[R]`, `max[R]` . . . 8
- 4.11 Literals . . . 9

5 Semantic Restrictions Beyond the Grammar **10**
- 5.1 Range restriction / safety . . . 10
- 5.2 Term linearity (`docs/translation1.md`) . . . 10
- 5.3 Arity . . . 10
- 5.4 Recursion — one specific pattern only . . . 10
- 5.5 Reserved SQL relation names . . . 11

6 EDB Relations and the Host Interface **12**

7 Reserved Keywords, and Rel Features Explicitly Not Supported **13**
- 7.1 Reserved keywords never used by any parser rule . . . 13
- 7.2 Operators lexed but never reachable from a parser rule . . . 13
- 7.3 Higher-level Rel features with no counterpart at all . . . 14

8 Error Taxonomy **15**

9 Worked Examples **16**

10 Appendix — Reference Grammar Files **17**

1 Purpose and Scope

Rel2SQL is a C++ tool (`rai/rel2sql`) that translates a *subset* of RelationalAI's Rel language into SQL. It is built with ANTLR4 for parsing, and follows the pipeline:

`Parse → RelASTBuilder → RelContextBuilder → Translator → (optional) Optimizer`

with three public entry points (`src/api/translate.h`): `GetSQLRel` (translate a whole program of `def` declarations into `CREATE OR REPLACE VIEW` statements), `GetSQLFromFormula` (translate a single formula into a `SELECT`), and `GetSQLFromExpr` (translate a single expression).

This document specifies **exactly which Rel constructs rel2sql's parser and translator accept**, as opposed to the full Rel language described in RelationalAI's official documentation (`rel.relational.ai/rel`). The two are **not the same language**: rel2sql implements a small, carefully bounded core — full applications, partial applications, quantifiers, boolean connectives, linear arithmetic comparisons, a handful of aggregates, and one specific pattern of linear recursion — sufficient to compile to standard SQL (including `WITH RECURSIVE` for the one recursion shape it recognizes). Most of Rel's richer surface (modules, integrity constraints, entities/value types, annotations, composition/restriction/override operators, general set algebra, `for`/`from`/`with`, varargs, etc.) is either rejected outright by the grammar or lexically reserved but never wired into a parser rule.

Method and provenance. Everything stated here was derived by reading the tool's own source: the ANTLR grammar files (`src/grammar/RelLexer.g4`, `src/grammar/RelParser.g4`), the AST node definitions (`src/rel_ast/rel_ast.h`), the AST builder (`src/rel_ast/rel_ast_builder.cc`), the safety/recursion/preprocessing passes (`src/preprocessing/`), the exception taxonomy (`src/support/exceptions.h`), the design notes in `docs/translation1.md`, and the test suite (`tests/test_translation.cc`, `tests/test_exceptions.cc`, `tests/test_safety.cc`), plus the example files under `examples/`. Every claim below is traceable to one of these files, and section 9 quotes concrete input/output pairs taken verbatim from the test suite as ground truth. Because this specification was reconstructed rather than published by the tool's authors, treat it as a snapshot of current behavior — re-derive it after any change to `src/grammar/` or `src/rel_ast/rel_ast_builder.cc`.

2 Program Structure

A rel2sql **program** is a sequence of zero or more definitions, each of the form:

```
def <Name> { <expr> ; <expr> ; ... }
```

There is no top-level statement other than `def`. A program is just `relDef*` followed by end-of-input — there are no standalone formulas, imports, or module headers at the program level (a single **formula** or **expression** can, however, be fed directly to the `GetSQLFromFormula` / `GetSQLFromExpr` entry points without being wrapped in a `def`, which is how `tests/test_translation.cc` and `examples/manual-formula-suite.rel` exercise the grammar).

Key structural facts:

- **A definition's head takes no parameters.** Unlike full Rel's `def foo(x, y) = ...` or `def foo[x] = ...`, rel2sql's `relDef` is exactly `'def' T_ID relAbs` — a bare name followed by a brace-delimited union of expressions. Any parameters must be introduced *inside* the body, via a binding form (`(x): ...` or `[x in D]: ...`; see §4.9).
- **The body is a union.** `relAbs` is `'{' expr (';' expr)* '}'` — one or more expressions separated by `;`, unioned together. A single-expression body is just `{expr}`.
- **A relation name may be defined more than once.** Multiple `def Name { ... }` clauses with the same `Name` are collected (`RelDef::multiple_defs` in `rel_ast.h`) and unioned. This is also the mechanism by which the one supported recursion pattern is expressed: a base-case `def` and a recursive-branch `def` sharing the same name (see §5.4), or simply two independent non-recursive clauses that happen to share a name.
- **Relation names cannot be reserved SQL keywords.** `def SELECT { 1 }` is rejected with a `SemanticException` (`ErrorCode::RESERVED_RELATION_NAME`) — see §5.5.
- **Undefined relations are rejected.** Referencing an identifier that is neither a `def`'d IDB nor a declared EDB throws `ArityException` ("relation ... not defined").
- **EDB (base) relations are supplied out-of-band**, not declared inside the Rel source text (there is no `bound`, `entity type`, or CSV-import syntax in this grammar at all). The host program passes a `RelationMap` — a `std::unordered_map<std::string, RelationInfo>` — mapping relation names to either a bare arity (columns are then named `A1, A2, ...`) or an explicit list of column names (`RelationInfo(std::vector<std::string> names)`), used verbatim as the underlying SQL table's column names. See §6.

3 Formal Grammar

This is a cleaned-up EBNF transcription of the productions actually present in `src/grammar/RelParser.g4` and the token definitions from `src/grammar/RelLexer.g4` that are reachable from those productions. Tokens that exist in the lexer but are **never referenced by any parser rule** are listed separately in §7 — the lexer reserves more surface than the parser accepts.

3.1 Top level

```
program   ::= relDef* EOF

relDef    ::= "def" Id relAbs

relAbs    ::= "{" expr (";" expr)* "}"
```

3.2 Expressions

```
expr        ::= literal                                  ; litExpr
              | "(" (expr ("," expr)*)? ")"               ; productExpr  (tuple / product)
              | term                                       ; termExpr
              | expr "where" formula                       ; conditionExpr
              | relAbs                                     ; relAbsExpr
              | formula                                    ; formulaExpr
              | "[" bindingInner "]" ":" expr              ; bindingsExpr
              | "(" bindingInner ")" ":" formula           ; bindingsFormula
              | applBase "[" applParams "]"                ; partialAppl
```

3.3 Formulas

Formula precedence, tightest to loosest binding (mirrors comments in the grammar: "atomic formulas first, then not, then and, then or"):

```
formula     ::= "{" ("(" ")")? "}"                         ; formulaBool     ({} = false, {()} = true)
              | applBase "(" applParams? ")"                ; FullAppl
              | "exists" "(" "(" bindingInner ")" "|" formula ")"  ; quantification
              | "forall" "(" "(" bindingInner ")" "|" formula ")"  ; quantification
              | "(" formula ")"                              ; paren
              | term comparator term                         ; comparison
              | "not" formula                                ; unOp
              | formula "and" formula                        ; binOp (left-assoc)
              | formula "or"  formula                        ; binOp (left-assoc)

comparator  ::= "=" | "!=" | "≠" | "<" | "<=" | "≤" | ">" | ">=" | "≥"
```

3.4 Terms (arithmetic — used only inside comparisons and as application arguments' numeric core)

```
term        ::= term ("*" | "/") term                     ; opTerm
              | term ("+" | "-") term                       ; opTerm
              | Id                                          ; IDTerm
              | numericalConstant                           ; numTerm
              | "(" term ")"                                ; parenthesisTerm
```

```
numericalConstant ::= IntLit | "-" IntLit | FloatLit | "-" FloatLit
```

Note what is conspicuously **absent** from `term`: no `^` (power), `%` (mod), `÷` (truncating division), and no unary minus on a general sub-term (only directly in front of an integer/float literal, as part of `numericalConstant`). `a - -b` or `-x` for a variable `x` is not a valid `term`.

3.5 Applications, bindings, literals

```
applBase    ::= Id | relAbs

applParam   ::= "_" | expr

applParams  ::= applParam ("," applParam)*

binding     ::= literal
              | Id ("in" Id)?

bindingInner ::= binding ("," binding)*

literal     ::= IntLit | "-" IntLit | MetaIntLit | FloatLit | "-" FloatLit
              | RelNameLit | RelNameStrLit | RelNameMultilineStrLit
              | CharLit | StringLit | MultilineStringLit | RawStringLit
              | DateLit | DateTimeLit
              | "true" | "false"
              | InterpolatedString
```

`Id` (`T_ID`) is a Unicode identifier (`[a-zA-Zα-ωA-Ω_][a-zA-Zα-ωA-Ω_0-9]*`, plus an optional value-type-style `^`-prefixed form); it is case-sensitive and cannot be one of the 21 keywords reserved by the lexer (§7.1).

4 Construct-by-Construct Semantics

Each accepted construct's meaning, illustrated with SQL rel2sql actually emits (all examples verified against `tests/test_translation.cc`).

4.1 Full application — `R(x, y, ...)`

A **formula**. True/false membership test, and simultaneously the primary way to *bind* fresh variables to a relation's columns (Datalog-style). `_` is a wildcard argument that binds nothing.

```
A(x)                →  SELECT T0.A1 AS x FROM A AS T0
R(x, x)         →  SELECT T0.id AS x FROM R AS T0 WHERE T0.id = T0.parent_id  (repeated var ⇒ self-
equality)
```

4.2 Partial application — `R[x]`, `R[1, y]`

An **expression**. Fixes a prefix of a relation's columns and returns the relation of remaining columns (arity `n − k` for `k` fixed args against an arity-`n` relation). Chains and nesting are supported (`B[E[x]]`, `C[B[x], y]`).

```
def S {R[x]}   (R named student_id, course_id, grade)
   →  CREATE OR REPLACE VIEW S AS (SELECT DISTINCT T0.student_id AS x,
        T0.course_id AS A1, T0.grade AS A2 FROM R AS T0);
```

The application base (`applBase`) may itself be an inline relational abstraction: `{A(x)}[x]`, `{(1,2);(3,4)}[1]`.

4.3 Conjunction / disjunction / negation

`and`, `or`, `not`, and the parenthesized form `(formula)`. Standard left-associative binding as in §3.3. Negation requires every free variable of the negated formula to already be *safe* (bound elsewhere in the enclosing conjunction) — see §5.1.

```
A(x) and B(x, y)   →  join on shared variable x
A(x) or  B(x)      →  UNION of the two branches
A(x) and not D(x)  →  SELECT ... WHERE NOT EXISTS (correlated on x)
```

4.4 Quantification — `exists`, `forall`

Syntax is fixed: `exists( (bindings) | formula )` / `forall( (bindings) | formula )` — note the **double parenthesization** (one pair around the whole quantifier call, one pair around the bindings) and the pipe `|` separating bindings from body. This is a different surface syntax from full Rel's `exists(bindings : formula)`.

```
(x, y): exists((z) | B(x, z) and E(z, y))
   →  SELECT T0.A1 AS A1, T1.A2 AS A2 FROM B AS T0, E AS T1 WHERE T0.A2 = T1.A1
```

(this is rel2sql's way of expressing relational **composition** — there is no `.` operator; see §7.2.)

`forall` compiles to a double-negation / anti-join shape; bindings may restrict a variable's domain with `in`: `exists((y in A) | B(x,y))`.

4.5 Comparisons

`term comparator term`, where `comparator ∈ {=, !=, <, >, <=, >=}` (plus the Unicode spellings `≠ ≤ ≥`). Both sides are arithmetic `terms` (§3.4), so comparisons can involve `+ - * /` combinations of identifiers and numeric literals: `x*x = 25`, `x + y > 10`, `(x + y) * z > 100`.

4.6 Conditional expression — `where`

`expr "where" formula` (an *expression*-level construct, not `if/then/else`). Per `docs/translation1.md`, `e where F` is understood as "the value of `e`, restricted to bindings where `F` holds," and is desugared to a filtered relational abstraction.

```
B[x] where A(x)                    →  {(z): z = B[x] and A(x)}, roughly
def R {[x] : x+1 where A(x)}       →  CREATE OR REPLACE VIEW R AS (SELECT DISTINCT
                                        T0.A1 AS A1, T0.A1 + 1 AS A2 FROM A AS T0);
```

Chained/nested `where` is supported: `(B[x] where A(x)) where D(x)`.

4.7 Tuples and products — `(e1, e2, ...)`

`"(" productInner ")"` builds a tuple/product expression. `(1, 2)` translates to a two-column constant row (`SELECT 1 AS A1, 2 AS A2`).

4.8 Relational abstraction (union) — `{e1; e2; ...}`

`relAbs` — a set literal / union of expressions, used both as a definition's body and as an inline anonymous relation anywhere an `applBase` or `expr` is expected. `{1}`, `{1;2}`, `{(1,2);(3,4)}` are all valid; each `;`-separated branch's columns are unioned (with column-alignment logic that can reorder or introduce a wrapping subquery when branches bind variables in different orders — see `UnionColumnAlignmentSwappedBindings` in the test suite).

4.9 Bindings — `(x): formula` and `[x in D]: expr`

Two binding forms exist, and they are **not interchangeable**:

- **Parenthesized bindings** `"(" bindingInner ")" ":" formula` — the body must be a *formula*; this is the idiomatic way to name free variables of a boolean query: `(x): A(x)`, `(x, y): A(x) and B(x,y)`.
- **Bracketed bindings** `"[" bindingInner "]" ":" expr` — the body is a general *expression* (so it may itself contain arithmetic, aggregation, or nested bindings): `[x in T, y in R]: F[x,y]`, `[x in A, y]: C[x,y] where D(y)`.

A binding variable may be domain-restricted with `in`: `x in A`. Constants are permitted as bindings too (`binding ::= literal | Id ("in" Id)?`), e.g. `x, 1: abc(x)`.

There is **no** `where`-clause-on-bindings, `from`, `for`, or `|`-as-comprehension form here (those exist in full Rel; see §7.3) — the only place `|` appears in this grammar is as the quantifier separator (§4.4).

4.10 Aggregation — `sum[R]`, `average[R]`, `min[R]`, `max[R]`

Aggregation has **no dedicated syntax**: `sum[A]` etc. parse as an ordinary partial application (`applBase "[" applParams "]"`) whose base identifier happens to be one of a fixed, hard-coded set of names recognized post-parse by the translator (`src/sql/aggregate_map.h`):

Rel identifier	SQL aggregate
`sum`	`SUM`
`average`	`AVG`
`avg`	`AVG`
`min`	`MIN`
`max`	`MAX`

This is the complete list. `count`, `argmax`, `argmin`, `mean`/`median`, and any other stdlib aggregate from full Rel are *not* recognized — using them as an identifier is legal syntax (it's just an unrecognized relation name)

but will fail as "undefined relation" unless the caller also happens to supply an EDB of that name. Grouped aggregation works by applying the aggregate to an expression that still has free (non-aggregated) variables, e.g. `def S {max[R[x]]}` compiles to a `GROUP BY x`.

4.11 Literals

Integers, floats (including a leading-- negative form folded directly into the literal), characters (`'a'`), plain and triple-quoted strings, booleans, dates (`2023-01-23`) and datetimes are all accepted by the grammar. **Important caveat:** only `int`, `float`, `bool`, and plain double-quoted string literals are given real semantic treatment (`RelASTBuilder::visitInt/visitFloat/visitBool/visitStr`, which parse the numeric value or strip the surrounding quotes). Relname literals (`:foo`), meta-int literals (`#17`), multi-line strings, raw strings, date and datetime literals, and interpolated strings are all captured as a `RelLiteral` whose value is **the raw, unprocessed source text** (`ctx->getText()` verbatim, including punctuation such as the leading `:` or the surrounding quotes) — i.e. the grammar accepts them, but nothing downstream interprets them as dates, symbols, or performs the string interpolation. Treat anything other than a bare `Int`, `Float`, `Bool`, or single-line double-quoted `String` literal as **parses, but semantics are not implemented / not reliable**.

5 Semantic Restrictions Beyond the Grammar

Parsing successfully is necessary but not sufficient — several whole-program analyses (`src/preprocessing/`) can still reject a syntactically valid program, or silently constrain what it can mean.

5.1 Range restriction / safety

`src/preprocessing/safety_inferrer.h` runs a fixpoint over the AST computing, for every subformula, the set of variables that are *safe* (i.e., provably bound to a finite set of values by some positive relational application in the enclosing conjunction). A variable used in a negation, a `forall`, or a comparison must already be safe from elsewhere in the same conjunction — an unsafe variable throws `QuantificationException` (`ErrorCode::UNSAFE_QUANTIFICATION`). This is the classical Datalog/relational-calculus **safety** (range-restriction) condition: `not A(x)` alone is unsafe, but `B(x) and not A(x)` is safe because `x` is bound by `B(x)`.

5.2 Term linearity (`docs/translation1.md`)

Wherever an arithmetic `term` appears as a **relational-application parameter** (`A(x+1)`) or on one side of an **equality with variables the tool must solve for** (`x = y + 1 and A(y)`), rel2sql requires the term to be a **non-null linear polynomial in at most one free variable** (affine: `a·x + b`). This is checked by a purely syntactic, sound-but-incomplete bottom-up pass (`term_polynomial_visitor`):

- Constants get `(a, b) = (0, c)`.
- A single variable gets `(a, b) = (1, 0)`; a term touching zero or more than one distinct variable is marked invalid for this analysis.
- `+`/`-` combine coefficients additively; `*` is only linear if at most one side depends on the variable; `/` is only linear if the divisor is a nonzero constant.
- The polynomial must not be identically zero (`a = 0 and b = 0` is rejected in parameter position), and its unique root `-b/a` (when `a ≠ 0`) is what the translator solves for.

The check is deliberately conservative: it *never* accepts a genuinely non-linear term, but it can reject a term that is linear only after algebraic simplification (e.g. `(x*x - x*x + x)/x`, which is `1` but is rejected because the surface syntax contains `x*x` and a division by `x`).

5.3 Arity

Every relation (EDB or IDB) has one fixed arity, inferred (for IDBs) from the shape of its `def` bodies or (for EDBs) supplied directly by the host in the `RelationMap`. Arity is checked at every full/partial application; mismatches throw `ArityException` (`ErrorCode::ARITY_MISMATCH`). Unlike full Rel, there is **no support for overloading one relation name across different arities or types** (full Rel's "multiple arities/types create an overloaded union" behavior, see the Rel Advanced Syntax reference, is not modeled here).

5.4 Recursion — one specific pattern only

`src/preprocessing/recursion_visitor.h/.cc` recognizes recursion only in a narrow, specific shape and compiles *only that shape* to a SQL `WITH RECURSIVE` common table expression. The pattern it looks for is a `def Q { ... }` (or a same-named pair of `def`s) whose body is (after flattening `or`) a union of:

1. One or more **base-case disjuncts** that do not call `Q` (directly or transitively through non-recursive IDBs), and
2. One or more **recursive disjuncts**, each containing **exactly one** self-referencing call to `Q`, either directly conjoined (`E(x,y) and Q(y)`) or wrapped in an `exists` (`exists((z) | R(x,z) and Q(z,y))`).

```
def Q {(x,y) : R(x,y) or exists((z) | R(x,z) and Q(z,y))}
   →  CREATE OR REPLACE VIEW Q AS (WITH RECURSIVE R0(A1, A2) AS (
        SELECT ... FROM R ... UNION SELECT ... FROM R, R0 WHERE ...
```

```
) SELECT DISTINCT R0.A1 AS A1, R0.A2 AS A2 FROM R0);
```

If a recursive branch contains **more than one** self-call (`def Q {(x): B(x) or (Q(x) and Q(x))}`), the pattern match fails and the tool does **not** emit `WITH RECURSIVE` — `tests/test_translation.cc`'s `RecursiveDisjunctTooManyCallsRejected` asserts exactly this: the emitted SQL is intentionally *not* a valid recursive program in that case (mutual/multi-way recursion, and any shape beyond "linear" recursion in this sense, is unsupported).

5.5 Reserved SQL relation names

A `def` whose name collides with a reserved SQL keyword (e.g. `SELECT`) is rejected with `ErrorCode::RESERVED_RELATION_NAME`, since IDB names become SQL view/CTE identifiers.

6 EDB Relations and the Host Interface

Rel2SQL has no in-language way to declare a base relation's schema — no `bound`, `entity type`, CSV import, or `insert`/`delete`. Instead, the **caller** supplies a `RelationMap` (`src/rel_ast/relation_info.h`), a map from relation name to a `RelationInfo`:

- `RelationInfo(int arity)` — an *unnamed* EDB; columns are auto-named `A1, A2, ..., A<arity>`.
- `RelationInfo(std::vector<std::string> names)` — a *named* EDB; the given names become the SQL column names directly (e.g. `RelationInfo({"student_id", "course_id", "grade"})`), and rel2sql aliases them to the query's Rel variable names in the generated `SELECT`.

Any identifier used in the Rel source that is not defined by a `def` and not present in this map is an **undefined relation** error. This means the "subset of Rel" rel2sql accepts is really a subset *parameterized by* whatever EDB schema the host chooses to declare for a given translation call — the grammar and semantics in this document are the invariant part; the EDB catalog is supplied per invocation (via the C++ API, or implicitly assumed by the CLI/tests).

7 Reserved Keywords, and Rel Features Explicitly Not Supported

7.1 Reserved keywords never used by any parser rule

`src/grammar/RelLexer.g4` defines the following keywords (so they can never be used as an identifier), yet **none of them appear in `RelParser.g4`** — meaning the corresponding Rel constructs are not merely limited, they are **entirely absent** from what this grammar can parse:

Keyword	Rel feature it belongs to (per full Rel)
`module`, `namespace`, `import`, `use`, `with`, `as`	Modules and imports
`ic`, `declare`, `requires`	Integrity constraints / declarations
`implies`, `iff`, `xor`	Additional logical connectives
`value` / `type` (soft keywords)	Value-type / entity-type declarations

Using any of `module`, `ic`, `with`, `implies`, etc. anywhere in a rel2sql program is a **parse error** — either because the keyword can't be used as an identifier, or because there is no grammar production that accepts it as a construct.

7.2 Operators lexed but never reachable from a parser rule

The lexer also defines tokens for a large family of operators that likewise never appear in `RelParser.g4`:

Token(s)	Would-be Rel meaning	Status
`.` (`T_OP_DOT`)	Composition / dot-join (`R.S`)	**Not supported.** Composition must instead be spelled out via `exists` (§4.4).
`<:`, `:>`	Prefix/suffix restriction	Not supported.
`<++`, `++>`	Left/right override	Not supported.
`++`	Concatenation	Not supported.
`∪`, `∩`, `×`	Explicit union/intersect/Cartesian-product operators	Not supported (`;` already gives union at the relational-abstraction level; there is no dedicated intersect or Cartesian-product operator).
`⊂ ⊃ ⊆ ⊇`	Subset/superset comparisons	Not supported.
`≡ ≢`	Relation-level (in)equivalence	Not supported.
`~= !~=`	"Loose" equality/inequality	Not supported.
`^` (power), `%` (mod), `÷` (truncating division)	Arithmetic	Lexed, but `term` (§3.4) only wires up `+ - * /`; using `^ % ÷` is a parse error.
`¬ → ← ⇒ ⇐ ⇔ ⇏` (Unicode logic symbols)	Alternate spellings of `not/implies/iff`	Not supported (only ASCII `not/and/or` plus the six comparators reach the grammar).
`@` (`T_AT_ID`)	Annotations (`@inline`, `@outline`, `@ondemand`, `@static`, `@function`)	Lexed as a token shape, never used — **no annotation syntax exists**.
`T_VARARGS_ID` (`x...`), `T_UNDERSCORE_VARARGS` (`_...`)	Varargs	Not supported.
`T_NS_ID` (`::a::b`)	Namespaced/module-qualified names	Not supported.

Token(s)	Would-be Rel meaning	Status
`T_DOCSTRING_LIT` (`doc"..."`)	Docstrings	Lexed, but not included in the `literal` rule — a parse error if used.

7.3 Higher-level Rel features with no counterpart at all

Beyond individual tokens, entire layers of full Rel have no representation in this grammar or AST:

- **Modules** (`module ... end`, parameterized modules, `with X use Y`) — no `RelModule`-like node exists in `rel_ast.h`.
- **Integrity constraints** (`ic ...`) — no `RelIC` node.
- **Entity types and value types** (`entity type`, `value type`, `^Constructor[...]`) — no constructor/entity node kind.
- **Annotations** (`@inline`, `@outline`, `@ondemand`, `@function`, `@static`) — none of the AST nodes carry an annotation field.
- **The `for` / `from` / `|`-as-comprehension binding forms** and general relational abstraction with a `where`-suffixed binding list (`x in D where P(x): e`) — only the two binding forms in §4.9 exist, and `|` is reserved for the quantifier syntax.
- **General higher-order definitions** (`@outline def F[R] = ...`) — a def's head is always a bare name; there is no way to parameterize a definition by a relation argument.
- **Recursion beyond the single linear pattern in §5.4** — no general fixpoint/stratification machinery; mutual recursion between two differently-named relations, or a branch with two or more recursive self-calls, is not translated as recursive.
- **CSV/JSON I/O, `load_csv`, `insert`/`delete`/`output`/`export`/`abort`/`rel:config` control relations** — rel2sql translates a fixed logical query to a fixed SQL view/select; there is no update or I/O layer.
- **The full aggregate library** — only `sum`, `average`/`avg`, `min`, `max` (§4.10); no `count`, `argmax`, `argmin`, `mean`, `median`, `top`, `bottom`, `sort`, etc.

8 Error Taxonomy

`src/support/exceptions.h` defines the error hierarchy every rejection falls into (each formatted as `[<Category> Error E<code>] <message>`, optionally with a source location and a caret-annotated code snippet):

Code	Name	Thrown when
E1	`SYNTAX_ERROR`	ANTLR parse failure (`ParseException`) — malformed input, unexpected/missing token.
E2	`UNEXPECTED_TOKEN`	(reserved for parser use)
E3	`MISSING_TOKEN`	(reserved for parser use)
E100	`UNDEFINED_RELATION`	An identifier is neither a `def` nor a supplied EDB (`ArityException`). Also used generically for arity-related semantic errors.
E101	`ARITY_MISMATCH`	A relation is applied with the wrong number of arguments.
E102	`UNBALANCED_VARIABLE`	Variable-binding inconsistency (`VariableException`).
E103	`UNSAFE_QUANTIFICATION`	Range-restriction / safety violation (`QuantificationException`, §5.1).
E104	`UNKNOWN_BINARY_OPERATOR`	Internal: unrecognized binary operator during translation.
E105	`UNKNOWN_UNARY_OPERATOR`	Internal: unrecognized unary operator during translation.
E106	`EXPECTED_SINGLE_PROJECTION`	A context expected exactly one projected column but found a different arity.
E107	`QUANTIFIED_VARIABLE_NOT_FOUND`	A quantifier binds a variable never used in its body.
E108	`UNKNOWN_QUANTIFICATION`	Internal: unrecognized quantifier kind.
E109	`RESERVED_RELATION_NAME`	`def` name collides with a reserved SQL keyword (§5.5).
E900	`INTERNAL_ERROR`	Internal invariant violation.
E901	`NOT_IMPLEMENTED`	A recognized-but-unimplemented code path (`NotImplementedException`).

9 Worked Examples

All examples below are taken **verbatim** from `tests/test_translation.cc` (test name given for traceability); the SQL shown is the tool's actual, exact output.

`EDBBindingFormula` — `(x): A(x)`

```
SELECT T0.A1 AS A1 FROM A AS T0
```

`BindingConjunction` — `(x, y): A(x) and B(x, y)`

```
SELECT T0.A1 AS A1, T1.A2 AS A2 FROM A AS T0, B AS T1 WHERE T0.A1 = T1.A1
```

`BindingDisjunction` — `(x): A(x) or B(x)`

```
SELECT T0.A1 AS A1 FROM A AS T0 UNION SELECT T1.A1 AS A1 FROM B AS T1
```

`Composition` (via `exists`) — `(x, y): exists((z) | B(x, z) and E(z, y))`

```
SELECT T0.A1 AS A1, T1.A2 AS A2 FROM B AS T0, E AS T1 WHERE T0.A2 = T1.A1
```

`Definition2` (`where`) — `def R {[x]: x+1 where A(x)}`

```
CREATE OR REPLACE VIEW R AS (SELECT DISTINCT T0.A1 AS A1, T0.A1 + 1 AS A2 FROM A AS T0);
```

`TableDefinition` — `def R {(1, 2); (3, 4)}`

```
CREATE OR REPLACE VIEW R AS (SELECT DISTINCT T0.A1 AS A1, T0.A2 AS A2
  FROM (VALUES (1, 2), (3, 4)) AS T0(A1, A2));
```

`NamedAttributesAggregate` — `def S {max[R[x]]}` (EDB `R` has named columns `student_id, grade`)

```
CREATE OR REPLACE VIEW S AS (SELECT DISTINCT T1.x, MAX(T1.A1) AS A1
  FROM (SELECT DISTINCT T0.student_id AS x, T0.grade AS A1 FROM R AS T0) AS T1
  GROUP BY T1.x);
```

`TransitiveClosure` — `def Q {(x,y) : R(x,y) or exists((z) | R(x,z) and Q(z,y))}`

```
CREATE OR REPLACE VIEW Q AS (WITH RECURSIVE R0(A1, A2) AS (
  SELECT T0.A1 AS x, T0.A2 AS y FROM R AS T0
  UNION
  SELECT T1.A1 AS x, T2.A2 AS y FROM R AS T1, R0 AS T2 WHERE T1.A2 = T2.A1
) SELECT DISTINCT R0.A1 AS A1, R0.A2 AS A2 FROM R0);
```

`RecursiveDisjunctTooManyCallsRejected` — **`def Q {(x): B(x) or (Q(x) and Q(x))}`** no `WITH RECURSIVE` is emitted — this recursion shape (two self-calls in one branch) is intentionally unsupported (§5.4).

`ExceptionTest.ReservedRelationName` — **`def SELECT { 1 }`** throws `SemanticException` — `[Semantic Error E109] ... reserved SQL keyword ... SELECT` (§5.5, §8).

10 Appendix — Reference Grammar Files

For readers who want the primary source rather than the cleaned-up transcription in §3, the two files that fully determine rel2sql's accepted syntax are:

- `src/grammar/RelLexer.g4` (342 lines) — token definitions, including the string-interpolation and raw-string lexer modes.
- `src/grammar/RelParser.g4` (95 lines) — the entire parser grammar: `program`, `relDef`, `literal`, `interpolationPart`, `bindingInner`, `binding`, `applBase`, `applParam`, `applParams`, `productInner`, `relAbs`, `comparator`, `numericalConstant`, `term`, `expr`, `formula`.

Because the parser grammar is only 95 lines long, it is reproduced here in full for convenient side-by-side reading against §3–§5 of this document:

```
parser grammar RelParser;

options { language = Cpp; tokenVocab = RelLexer; }

program: relDef* EOF;
relDef: s = 'def' name = T_ID relAbs;

literal:
    T_INT_LIT                                                  # int
  | T_OP_MINUS T_INT_LIT                                        # negInt
  | T_META_INT_LIT                                              # metaInt
  | T_FLOAT_LIT                                                 # float
  | T_OP_MINUS T_FLOAT_LIT                                      # negFloat
  | T_RELNAME_LIT                                               # relName
  | T_RELNAME_STR_LIT                                           # relNameStr
  | T_RELNAME_MSTR_LIT                                          # relNameMstr
  | T_CHAR_LIT                                                  # Char
  | T_STATIC_STR_LIT                                            # Str
  | T_STATIC_MSTR_LIT                                           # Mstr
  | T_RAWSTRING_LIT                                             # Rawstr
  | T_DATE_LIT                                                  # Date
  | T_DATETIME_LIT                                              # Datetime
  | meh = ('true' | 'false')                                    # bool
  | T_QUOTE (interpolationPart | ERR_INVALID_STR_LIT)+ T_QUOTE  # interpol
  | T_TRIPLE_QUOTE (interpolationPart | ERR_INVALID_STR_LIT)+ T_TRIPLE_QUOTE # interpol;

interpolationPart:
    T_STATIC_STR_PART
  | T_STATIC_MSTR_PART T_MIDDLE_QUOTES?
  | T_MIDDLE_QUOTES? T_STATIC_MSTR_PART
  | T_INTERPOL_ID
  | T_INTERPOL_START expr ')';

bindingInner: bindings = binding (',' bindings = binding)*;
binding: literal | id = T_ID ('in' id_domain = T_ID)?;

applBase: T_ID | relAbs;
applParam: underscore = '_' | expr;
applParams: params = applParam (',' params = applParam)*;
productInner: (exprs = expr (',' exprs = expr)*)?;
relAbs: '{' (exprs = expr (';' exprs = expr)*)? '}';
```

```
comparator: T_OP_COMP | T_OP_NEQ | T_OP_EQ;

numericalConstant:
    T_INT_LIT                                                   # numInt
  | T_OP_MINUS T_INT_LIT                                        # numNegInt
  | T_FLOAT_LIT                                                 # numFloat
  | T_OP_MINUS T_FLOAT_LIT                                      # numNegFloat;

// Operator precedence: first alternative = highest (binds tightest). So * / before + -.
term:
    lhs = term op = (T_OP_MULT | T_OP_DIV) rhs = term          # opTerm
  | lhs = term op = (T_OP_PLUS | T_OP_MINUS) rhs = term         # opTerm
  | T_ID                                                        # IDTerm
  | numericalConstant                                           # numTerm
  | '(' term ')'                                                # parenthesisTerm;

// The order of the rules below matters for precedence.
expr:
    literal                                                     # litExpr
  | '(' productInner ')'                                        # productExpr
  | term                                                        # termExpr
  | lhs = expr 'where' rhs = formula                            # conditionExpr
  | relAbs                                                      # relAbsExpr
  | formula                                                     # formulaExpr
  | '[' bindingInner ']' ':' expr                               # bindingsExpr
  | '(' bindingInner ')' ':' formula                            # bindingsFormula
  | applBase '[' applParams ']'                                 # partialAppl;

// Formula with operator precedence: not > and > or.
// Order matters: atomic formulas first, then not, then and, then or.
formula:
    '{' ('(' ')')? '}'                                          # formulaBool
  | applBase '(' applParams? ')'                                # FullAppl
  | op = 'exists' '(' '(' bindingInner ')' '|' formula ')'      # quantification
  | op = 'forall' '(' '(' bindingInner ')' '|' formula ')'      # quantification
  | '(' formula ')'                                             # paren
  | lhs = term comparator rhs = term                            # comparison
  | op = 'not' formula                                          # unOp
  | lhs = formula op = 'and' rhs = formula                      # binOp
  | lhs = formula op = 'or' rhs = formula                       # binOp;
```